

Capstone Project Report: CollabDoc (Real-Time Collaboration Platform)

Course / Capstone Submission

Project Title: CollabDoc — Real-Time Collaboration Platform

Team Members:

- **Shivam Mittal** (Backend Engineering, DevOps)
 - **Divya Singh** (System Design, Software Engineering, Concurrency Core)
 - **Kartik Yadav** (Frontend Engineering)
 - **Hardik Hathwal** (Backend Engineering , System Design)
 - **Kabir Sharma** (Frontend Engineering)
-

Executive Summary

This report outlines the structural, architectural, and logical breakdown of our capstone project, **CollabDoc**. The system is a highly concurrent, real-time document collaboration platform (similar to Google Docs) built from scratch. Our core achievement in this project is the custom implementation of **Operational Transformation (OT)** algorithms to resolve concurrent editing conflicts without relying on polling or locks.

1. System Architecture & Tech Stack

CollabDoc employs a distributed, highly concurrent Client-Server microservices architecture designed around OT.

- **Frontend:** Next.js 14, React, Tailwind CSS, TypeScript.
- **Backend:** Python 3.11+, FastAPI (Uvicorn), WebSockets.
- **Database:** PostgreSQL via `asyncpg` , managed by SQLAlchemy & Alembic.
- **Caching & Message Broker:** Redis (Pub/Sub) for horizontally scaling WebSocket connections across multiple Uvicorn worker nodes.

2. Backend Engine Analysis (/backend)

The backend is built around FastAPI and strictly follows SOLID principles and Gang of Four (GoF) design patterns to ensure enterprise-grade maintainability.

2.1 Entry Point and Routing

The FastAPI application configures CORS to accept traffic from the Next.js frontend. The API routes are strictly segregated:

- `routers.auth` : Handles JWT issuing and user login.
- `routers.documents` : Handles CRUD for documents via standard REST.
- `routers.websocket` : Exposes the WS duplex tunnel for real-time OT data streams.

2.2 Data Layer & Persistence

We utilized SQLAlchemy 2.0 with asynchronous drivers. The schema design is deeply normalized:

- **User** : Stores identity and credentials (`email` , `hashed_password`).

- **Document** : The central entity, tracking the current global `revision` , raw `content` , and ownership.
- **DocumentPermission** : Manages Role-Based Access Control (RBAC), implementing roles such as `owner` , `editor` , and `viewer` .
- **OperationLog** : Vaults every single keystroke as an individual OT (`op_type` , `position` , `char` , `length` , `revision`). This acts as an event-sourcing log, crucial for point-in-time recovery and history playback.
- **Version** : Stores materialized snapshots to avoid re-running thousands of operations to rebuild the document state from scratch.

2.3 Operational Transformation (OT) Engine

This module is the mathematical core of our platform, solving race conditions between concurrent users typing at the exact same time.

- **Command Pattern (`operation.py`)**: Defines classes like `InsertOperation` and `DeleteOperation` . Each encapsulates an atomic action and provides a polymorphic `.apply(content)` method.
- **Strategy Pattern (`transformer.py`)**: The `Transformer` class contains static methods representing the complex matrix of concurrent state changes. For example:
 - `_transform_insert_insert` : Resolves ties using deterministic `user_id` fallback.
 - `_transform_insert_delete` & `_transform_delete_insert` : Adjusts cursor positions based on character offsets.
 - `_transform_delete_delete` : A complex 6-case logic tree handling overlapping deletion ranges (e.g., partial overlaps, subsets).

2.4 Application Services

We utilized the **Facade Pattern** to decouple and hide complex SQL transaction logic from the FastAPI Routers.

- **`ot_service.py`** : Interacts with the `Transformer` to safely append incoming keystrokes against the latest database revision, ensuring the database is the ultimate source of truth.
- **`auth_service.py`** : Manages bcrypt hashing and JWT generation.
- **`permission_service.py`** : Enforces the RBAC policies defined in the database across all endpoints.

3. Frontend Architecture (`/frontend`)

The Next.js 14 App Router drives the frontend, focusing on low latency and responsive user experience.

3.1 Editor Core Component

The editor is a complex React component that manages the document's real-time state independently of the server.

- **State Management**: It maintains an internal `contentRef` and `revisionRef` to stay perfectly synchronized with the server without suffering from React re-render lag.
- **WebSocket Tunnel (`CollabWebSocket`)**: Instantiated on mount, this handles the `ws://` protocol. It binds to event listeners for `init` (snapshot), `ack` (receipt of our operations), `operation` (remote keystrokes), and `cursor` (remote typing indicators).
- **Local Diffing & OT Generation**: When the user types, the component calculates the difference between the old and new text locally, generating OT commands to broadcast over the socket.

- **Cursor Remapping:** Ensures the user's local cursor does not jump wildly when remote operations shift the text index.
- **Throttling:** Cursor updates are actively throttled (`CURSOR_THROTTLE_MS = 200`) to prevent overwhelming the server.

3.2 Dynamic Component Loading

Non-critical UI panels (like Version History, Share menus, and AI features) are heavily optimized. They are wrapped in `next/dynamic` with `ssr: false`, meaning their JavaScript chunks are entirely deferred until the user actually clicks their respective buttons, drastically reducing initial load time.

4. Assessment & Conclusion

Through this capstone project, our group successfully implemented a highly complex distributed system.

1. **Concurrency Handling:** By abstracting operations into mathematical transforms and utilizing Redis Pub/Sub, the system is highly resilient to race conditions and horizontal scaling issues.
2. **Security:** WebSockets require token authentication on connection initialization, and `permission_service.py` strictly gates document access.
3. **Performance:** The frontend debounces and throttles network-heavy events, while the backend utilizes Python's `asyncio` top-to-bottom for non-blocking I/O.
4. **Extensibility:** Our adherence to GoF principles (specifically the open/closed nature of the `Operation` base class) means that extending this project to support rich text formatting (e.g., bold, italic) would only require adding new `Operation` subclasses without modifying the core `Transformer` engine.

Conclusion: CollabDoc stands as an exceptionally well-engineered implementation of a distributed real-time collaborative architecture, fulfilling and exceeding the requirements set out for this capstone project.